Workbook 2: Object Orientated Programming Inheritance, Polymorphism and Overriding.

Please complete the tasks in this workbook.

For any code created a screen shot should be provided in this document and the code file saved as a .py file.

Each section or task will need its own code file.

When complete save it as a PDF file with the following naming convention and then submit:

oop2_<student number>_<student name>.pdf

For example:

oop2_123456_fred_blogs.pdf

Task 1

Extending our person Class from the previous worksheet.

A college has many students.

Each student will have their own record.

Some new software has been installed but it is not quite complete and is missing a way of displaying a student record. You have been asked to rectify this and add some more functionality.

The Following code has been provided:

```
class Person():
    def __init__(self, age, name):
        self.age = age
        self.name = name

    def birthday(self):
        self.age = self.age + 1


ben = Person(31, "Ben")
paul = Person(42, "Paul")
print(ben.name, ben.age)
print(paul.name, paul.age)
```

This shows a class, called *Person*, it's constructor (this will be explained in the next lesson) used to setup the object and a single method, *birthday(),* to adjust the birthday value.

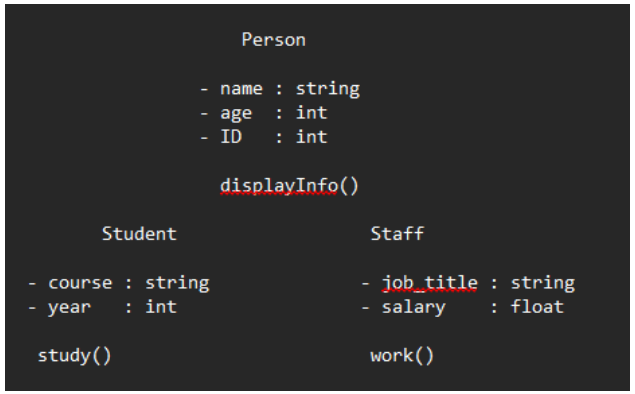
Some example code is also provided on how to use the class.

You will need to copy the above code into the IDE of your choice and make sure it works before attempting the tasks below.

Task 1.A:

Produce a design on paper to enable the Person class to be a super class along with classes that create student and staff objects.

1.A Answer:



Task 1.B

Convert this design into software and show the output below along with a copy of the software.

1.B Answer:

```
"""
Person class to be a super class.
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
# Superclass
class Person:
    def __init__(self, name, age, ID):
        self.name = name
        self.age = age
        self.ID = ID

    def displayInfo(self):
        print(f"Name: {self.name}, Age: {self.age}, ID: {self.ID}")


# Classes
class Student(Person):
    def __init__(self, name, age, ID, course,):
        super().__init__(name, age, ID)
        self.course = course


    def study(self):
        print(self.name + " is studying " + self.course)


class Staff(Person):
    def __init__(self, name, age, ID, job_title,):
        super().__init__(name, age, ID)
        self.job_title = job_title

    def work(self):
        print(self.name + " is working as a " + self.job_title)

# Code
student1 = Student("Ayaan", 16, 298496, "Software Development Student")
staff1   = Staff("Michael", 42, 123456, "Software Development Teacher")

student1.displayInfo()
student1.study()

print()

staff1.displayInfo()
staff1.work()
```

```
Name: Ayaan, Age: 16, ID: 298496
Ayaan is studying Software Development Student

Name: Michael, Age: 42, ID: 123456
Michael is working as a Software Development Teacher
```

Software Development OOP Workbook 2

Task 1.C

On the previous worksheet you were asked to create a method to save data to a file.

Can you now create a class that will handle this process along with being able to read CSV data from a file and use it to create records for students and staff.

1.C Answer:

```
"""
Handling classes with data from CSV file.
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
# imports
import csv
import os

# Superclass
class Person:
    def __init__(self, name, age, ID):
        self.name = name
        self.age = age
        self.ID = ID

    def displayInfo(self):
        print(f"Name: {self.name}, Age: {self.age}, ID: {self.ID}")

# Classes
class Student(Person):
    def __init__(self, name, age, ID, course):
        super().__init__(name, age, ID)
        self.course = course

    def study(self):
        print(self.name + " is studying " + self.course)


class Staff(Person):
    def __init__(self, name, age, ID, job_title):
        super().__init__(name, age, ID)
        self.job_title = job_title

    def work(self):
        print(self.name + " is working as a " + self.job_title)

# Manager class
class RecordManager:
    def __init__(self):
        self.records = []

    def load_from_csv(self, filename):

        with open(filename, newline='') as file:
            reader = csv.reader(file)
            next(reader)

            for row in reader:
                if len(row) < 5:
                    continue

                p_type, name, age, ID, field5= row

                age = int(age)
                ID = int(ID)

                if p_type.lower() == "student":
                    new_obj = Student(name, age, ID, field5)
                elif p_type.lower() == "staff":
                    new_obj = Staff(name, age, ID, field5)
                else:
                    continue
                self.records.append(new_obj)
# Code
    def show_all(self):
        for person in self.records:
            person.displayInfo()
            if isinstance(person, Student):
                print(" Role: Student | Course:", person.course)
            elif isinstance(person, Staff):
                print(" Role: Staff | Job:", person.job_title)
            print()

manager = RecordManager()
manager.load_from_csv("people.csv")
manager.show_all()
```

type	name	age	id	course_or_job
student	Ayaan	18	298496	Software Development
staff	Michael	42	123456	Software Development Teacher
student	Artin	17	212422	Cyber Security
staff	Richard	40	221198	Software Development Teacher

```
Name: Ayaan, Age: 18, ID: 298496
 Role: Student | Course: Software Development

Name: Michael, Age: 42, ID: 123456
 Role: Staff | Job: Software Development Teacher

Name: Artin, Age: 17, ID: 212422
 Role: Student | Course: Cyber Security

Name: Richard, Age: 40, ID: 221198
 Role: Staff | Job: Software Development Teacher
```

Task 2:

Pizza Code Express

Read the attached Pizza Code assignment.

Produce two separate versions of it:

- One in the procedural paradigm
- One in the OOP paradigm

Along with your solution answer the following questions:

1. What is a Paradigm in respect to Programming? (Provide some examples.) Programming paradigms are different ways or styles in which a given program or programming language can be organized. Each paradigm consists of certain structures, features, and opinions about how common programming problems should be tackled.
2. Which have you found better (OOP or Procedural) to write your solution in? OOP
3. Why? Because I could split the program into different classes, like pizzas, orders, and the menu. This made it easier to keep track of everything and made the code less messy compared to just writing it all as functions. It also makes it easier to add new features later.

Task 2 Answer:

```
"""
Phase 1 of Calder Pizza Company
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
# Variables
branch = "Calder Pizza Company."
print("Hello and Welcome to", branch)
total = 0
price_small = 5.50
price_medium = 7.90
price_large = 12.00
price_topping = 0.75
available_toppings = [
    "Pepperoni", "Mushroom", "Extra cheese", "Sausage",
    "Onion", "Black olives", "Green pepper", "Fresh garlic",
    "Tomato", "Fresh basil"
]
small_orders = []
medium_orders = []
large_orders = []

# Code.
while True:

# Ask for pizza size.
    while True:
        size = input("What size pizza do you want? (small, medium, large): ").lower()
        if size in ["small", "medium", "large"]:
            break
        print("Please enter small, medium or large.")

    if size == "small":
        price = price_small
    elif size == "medium":
        price = price_medium
    else:
        price = price_large

# Ask for number of toppings.
    while True:
        toppings_count = input("How many toppings do you want? (0-3): ")
        if toppings_count.isdigit():
            toppings_count = int(toppings_count)
            if 0 <= toppings_count <= 3:
                break
        print("Enter a number between 0 and 3.")

# Choose toppings.
    toppings = []

    if toppings_count > 0:
        print("\nAvailable toppings:")
        for i in range(len(available_toppings)):
            print(f"{i + 1}. {available_toppings[i]}")

        for i in range(toppings_count):
            while True:
                choice = input(f"Choose topping {i + 1}: ")
                if choice.isdigit():
                    choice = int(choice)
                    if 1 <= choice <= len(available_toppings):
                        toppings.append(available_toppings[choice - 1])
                        break
                print("Invalid choice.")

# Calculate cost of pizza.
    pizza_cost = price + (toppings_count * price_topping)
    total += pizza_cost

# Storing the order.
    order = {
        "size": size,
        "toppings": toppings,
        "cost": pizza_cost
    }
    if size == "small":
        small_orders.append(order)
    elif size == "medium":
        medium_orders.append(order)
    else:
        large_orders.append(order)

# Output order details.
    print("\nYour Order")
    print("Size:", size)
    print("Toppings:", ", ".join(toppings) if toppings else "None")
    print(f"Pizza cost: £{pizza_cost:.2f}")
    print(f"Total so far: £{total:.2f}")
    print()

# Ask to continue.
    again = input("Do you want to order another pizza? (y/n): ").lower()
    if again != "y":
        break

# Final order summary
print("\nFinal Order Summary")
print("Small pizzas:", len(small_orders))
print("Medium pizzas:", len(medium_orders))
print("Large pizzas:", len(large_orders))
print(f"Grand Total: £{total:.2f}")
print("Thank you for your order!")
```

- Procedural paradigm.

Software Development OOP Workbook 2

```
Hello and Welcome to Calder Pizza Company.
What size pizza do you want? (small, medium, large): medium
How many toppings do you want? (0-3): 1

Available toppings:
1. Pepperoni
2. Mushroom
3. Extra cheese
4. Sausage
5. Onion
6. Black olives
7. Green pepper
8. Fresh garlic
9. Tomato
10. Fresh basil
Choose topping 1: 2

Your Order
Size: medium
Toppings: Mushroom
Pizza cost: £8.65
Total so far: £8.65

Do you want to order another pizza? (y/n): n

Final Order Summary
Small pizzas: 0
Medium pizzas: 1
Large pizzas: 0
Grand Total: £8.65
Thank you for your order!
```

```
"""
Phase 1 of Calder Pizza Company.
"""
__author__ = "298496 Ayaan Shahzad"
__version__ = "0.0.1"
__license__ = "MIT"
class Menu:
    def __init__(self):
        self.price_small = 5.50
        self.price_medium = 7.90
        self.price_large = 12.00
        self.price_per_topping = 0.75

        self.available_toppings = [
            "Pepperoni", "Mushroom", "Extra cheese", "Sausage",
            "Onion", "Black olives", "Green pepper", "Fresh garlic",
            "Tomato", "Fresh basil"
        ]

    def get_price(self, size):
        if size == "small":
            return self.price_small
        elif size == "medium":
            return self.price_medium
        else:
            return self.price_large


class Pizza:
    def __init__(self, size, toppings, cost):
        self.size = size
        self.toppings = toppings
        self.cost = cost

class Order:
    def __init__(self):
        self.small = []
        self.medium = []
        self.large = []
        self.total = 0

    def add_pizza(self, pizza):
        if pizza.size == "small":
            self.small.append(pizza)
        elif pizza.size == "medium":
            self.medium.append(pizza)
        else:
            self.large.append(pizza)

        self.total += pizza.cost

    def show_summary(self):
        print("\nFinal Order Summary")
        print("Small pizzas ordered:", len(self.small))
        print("Medium pizzas ordered:", len(self.medium))
        print("Large pizzas ordered:", len(self.large))
        print(f"Grand Total: £{self.total:.2f}")
        print("Thank you for your order!")
        print()
```

```
class PizzaShop:
    def __init__(self):
        self.branch = "Calder Pizza Company."
        self.menu = Menu()
        self.order = Order()

    def display_welcome_message(self):
        print(f"Hello and Welcome to {self.branch}")

    def ask_size(self):
        size = input("What size pizza do you want? (small, medium, large): ").lower()
        if size in ["small", "medium", "large"]:
            return size
        print("Not valid. Please enter small, medium or large.")
        return self.ask_size()

    def ask_topping_count(self):
        num = input("How many toppings do you want? A maximum of 3 please: ")
        if num.isdigit():
            num = int(num)
            if 0 <= num <= 3:
                return num
        print("Enter a number from 0 to 3 please.")
        return self.ask_topping_count()

    def ask_single_topping(self, num):
        choice = input(f"Topping {num}: ")
        if choice.isdigit():
            choice = int(choice)
            if 1 <= choice <= len(self.menu.available_toppings):
                return self.menu.available_toppings[choice - 1]
        print("Invalid choice.")
        return self.ask_single_topping(num)

    def choose_toppings(self, count):
        toppings = []

        if count > 0:
            print("\nAvailable toppings:")
            for i in range(len(self.menu.available_toppings)):
                print(f"{i + 1}. {self.menu.available_toppings[i]}", end="    ")
            print("\n")

            for i in range(1, count + 1):
                toppings.append(self.ask_single_topping(i))

        return toppings

    def display_pizza(self, pizza):
        print("\nYour Order Details")
        print("Size:", pizza.size.capitalize())
        print("Toppings:", ", ".join(pizza.toppings) if pizza.toppings else "None")
        print(f"Pizza cost: £{pizza.cost:.2f}")
        print(f"Total: £{self.order.total:.2f}")
        print()

    def get_pizza_order(self):
        size = self.ask_size()
        topping_count = self.ask_topping_count()
        toppings = self.choose_toppings(topping_count)
        base_price = self.menu.get_price(size)
        cost = base_price + topping_count * self.menu.price_per_topping

        pizza = Pizza(size, toppings, cost)
        self.order.add_pizza(pizza)
        self.display_pizza(pizza)

    def ask_to_continue(self):
        while True:
            answer = input("Do you want to restart? (y/n): ").lower()
            if answer == "y":
                print("\nProgram restarting...\n")
                return True
            elif answer == "n":
                print("\nProgram has ended. Goodbye!\n")
                return False
            else:
                print("Not valid.")

    def start(self):
        self.display_welcome_message()
        print()
        self.get_pizza_order()
        self.order.show_summary()


class Program:
    def run(self):
        shop = PizzaShop()
        while True:
            shop.start()
            if not shop.ask_to_continue():
                break

# Code.
Program().run()
```

Software Development OOP Workbook 2

OOP paradigm.